

**LAPORAN PRAKTIKUM
TUGAS 2**

**PERANCANGAN ENDPOINT API DAN
DOKUMENTASI OPENAPI/SWAGGER
STUDI KASUS APLIKASI WISATAKU**



Disusun oleh:
Baiq Agestia Cahya Ilami
NIM: TI21240005

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TINGGI MANAJEMEN INFORMATIKA DAN KOMPUTER
STMIK LOMBOK
PRAYA
2026**

1. Pendahuluan

Tugas 2 (Bab 2 modul) merancang endpoint API WisataKu dan menyusun dokumentasinya sesuai standar OpenAPI. Rancangan pada laporan ini bukan dokumen terpisah dari kode: seluruh endpoint, DTO, dan status code diambil dari aplikasi wisataku-api yang sudah berjalan, dan dokumen OpenAPI diekspor langsung dari decorator di kode.

Luaran	File
Rancangan endpoint	laporan ini, docs/tugas-2/api-design.md
OpenAPI JSON	docs/tugas-2/openapi.json
OpenAPI YAML	docs/tugas-2/openapi.yaml
Catatan verifikasi Swagger	docs/tugas-2/swagger-verification.md

2. Konvensi URI

Mengikuti Bab 2.3.1 modul:

Aturan	Penerapan
Kata benda, bukan kata kerja	/destinasi, bukan /getDestinasi
Huruf kecil	/destinasi, /reservasi, /auth/login
Hierarki untuk relasi	/destinasi/:id/ulasan, /destinasi/:id/fasilitas
Filter dan pagination lewat query string	/destinasi?kategori=Pantai&page=1&limit=10
Aksi pada resource lewat method HTTP	POST /destinasi membuat, PATCH /destinasi/:id mengubah

Nama resource memakai bahasa Indonesia sesuai modul. Pengecualian kata kerja hanya pada /auth/register dan /auth/login, karena keduanya adalah operasi autentikasi, bukan resource.

3. Daftar Endpoint

Method	URI	Deskripsi	Akses	Request	Respons sukses
POST	/auth/register	Registrasi akun wisatawan	Publik	RegisterDto	201 RegisterResponse

Method	URI	Deskripsi	Akses	Request	Respons sukses
POST	/auth/login	Login, menerima JWT	Publik	LoginDto	200 LoginResponse
GET	/destinasi	Daftar destinasi, filter kategori, pagination	Publik	query kategori, page, limit	200 Destinasi[]
GET	/destinasi/:id	Detail destinasi	Publik	path id (integer)	200 Destinasi
GET	/destinasi/:id/ulasan	Ulasan destinasi, terbaru lebih dulu	Publik	path id	200 Ulasan[]
GET	/destinasi/:id/fasilitas	Fasilitas destinasi	Publik	path id	200 Fasilitas[]
POST	/destinasi	Tambah destinasi	Admin	CreateDestinasiDto	201 Destinasi
PATCH	/destinasi/:id	Ubah sebagian data destinasi	Admin	UpdateDestinasiDto	200 Destinasi
DELETE	/destinasi/:id	Hapus destinasi	Admin	path id	200 { message }
POST	/reservasi	Buat reservasi tiket	Wisatawan	CreateReservasiDto	201 Reservasi

Enam endpoint destinasi pertama sama dengan rancangan pada tabel Bab 2.3.1 modul. Endpoint fasilitas, auth, dan reservasi ditambahkan karena dibutuhkan oleh Bab 4 (perbandingan REST vs GraphQL), Bab 5, dan Bab 8.

DELETE membalas 200 dengan pesan konfirmasi, bukan 204, agar client menerima konfirmasi yang dapat ditampilkan. POST /auth/login memakai @HttpCode(200) karena login tidak membuat resource baru; status ini juga yang diharapkan skenario e2e Bab 8.4.

4. Status Code per Endpoint

Endpoint	200	201	400	401	403	404	409
POST /auth/register		✓	✓				✓ email terdaftar
POST /auth/login	✓		✓	✓ kredensial salah			
GET /destinasi	✓		✓ query tidak valid				
GET /destinasi/:id	✓		✓ id bukan angka			✓	
GET /destinasi/:id/ulasan	✓					✓	
GET /destinasi/:id/fasilitas	✓					✓	
POST /destinasi		✓	✓	✓	✓ bukan admin		
PATCH /destinasi/:id	✓		✓	✓	✓	✓	
DELETE /destinasi/:id	✓			✓	✓	✓	✓ sudah ada reservasi
POST /reservasi		✓	✓ tanggal lampau	✓	✓ bukan wisatawan	✓ destinasi	

Tabel ini sama dengan respons yang dideklarasikan di `openapi.json`. Status code tersebut juga diuji oleh e2e test (`test/destinasi.e2e-spec.ts`), yang lulus 41/41 pada run pengujian 30 September 2026 (output: `docs/tugas-4/test-results/e2e-test-2026-09-30.txt`).

5. Request dan Response

5.1 DTO dan validasi

DTO didefinisikan sebagai kelas dengan decorator `class-validator` dan `@ApiProperty`. Dengan begitu satu kelas menjadi kontrak validasi sekaligus skema OpenAPI.

ValidationPipe({ whitelist: true, transform: true }) dipasang global: field yang tidak dikenal dibuang, dan query string dikonversi ke tipe yang benar.

DTO	Field	Aturan
CreateDestinasiDto	nama	string, wajib, maks 150
	kategori	string, wajib, maks 50; kategori baru dibuat otomatis
	lokasi, deskripsi	opsional
	hargaTiket	number ≥ 0 , wajib
UpdateDestinasiDto	semua field CreateDestinasiDto	opsional (PartialType)
QueryDestinasiDto	kategori	opsional
	page	integer ≥ 1
	limit	integer 1–100; tanpa limit seluruh data dikembalikan
RegisterDto	nama	string, wajib, maks 100
	email	format email, maks 150
	password	8–72 karakter (72 adalah batas input bcrypt)
LoginDto	email, password	wajib
CreateReservasiDto	destinasiId	integer ≥ 1
	tanggalkunjungan	YYYY-MM-DD, tidak boleh sebelum hari ini
	jumlahTiket	integer 1–20

totalHarga pada reservasi tidak diterima dari client; server menghitungnya dari hargaTiket di database dikali jumlahTiket. Parameter :id divalidasi dengan ParseIntPipe, sehingga GET /destinasi/abc langsung ditolak 400 sebelum menyentuh database.

5.2 Contoh request dan response

Request membuat destinasi (admin):

```
POST /destinasi
Authorization: Bearer <token admin>
Content-Type: application/json

{ "nama": "Pantai Kuta Mandalika", "kategori": "Pantai", "lokasi": "Kuta, Lombok Tengah", "hargaTiket": 15000 }
```

Skema respons Destinasi (dari openapi.json): id, nama, kategori, lokasi, deskripsi, hargaTiket, ratingRata, createdAt.

5.3 Format error

Semua error memakai format bawaan NestJS, didokumentasikan di Swagger sebagai skema ErrorResponse:

```
{ "statusCode": 404, "message": "Destinasi dengan id 999 tidak ditemukan",  
  "error": "Not Found" }
```

Untuk error validasi, message berupa array pesan:

```
{ "statusCode": 400, "message": ["hargaTiket must not be less than 0"],  
  "error": "Bad Request" }
```

Karena semua respons error memakai skema ErrorResponse yang sama, contoh bawaannya semula selalu berupa body 404, termasuk pada respons 400, 401, 403, dan 409. Kekeliruan dokumentasi ini diperbaiki di buildOpenApiDocument (libs/common/src/http/configure-http-app.ts): setiap respons error kini diberi contoh sesuai kode statusnya, dengan pesan yang diambil dari respons asli API (misalnya "Validation failed (numeric string is expected)" untuk GET /destinasi/abc). Perubahan ini hanya menyentuh dokumen OpenAPI; perilaku API tidak berubah, dan unit test 32/32 serta e2e test 41/41 tetap lulus setelah perubahan.

6. Autentikasi pada Dokumentasi

DocumentBuilder().addBearerAuth() menambahkan skema keamanan bearer. Endpoint yang dilindungi diberi @ApiBearerAuth(), sehingga Swagger UI menampilkan ikon gembok dan tombol **Authorize** untuk memasukkan token hasil POST /auth/login. Pada openapi.json, empat operasi memiliki security: [{ bearer: [] }]: POST /destinasi, PATCH /destinasi/{id}, DELETE /destinasi/{id}, dan POST /reservasi.

7. Dokumen OpenAPI

File / URL	Isi
docs/tugas-2/openapi.json	OpenAPI 3.0.0, "WisataKu API" versi 1.0, 7 path / 10 operasi, 12 skema
docs/tugas-2/openapi.yaml	Isi sama dalam format YAML
http://localhost:3000/api/docs	Swagger UI, saat aplikasi berjalan

File / URL	Isi
http://localhost:3000/api/docs-json, /api/docs-yaml	Dokumen yang sama dari server

Modul Bab 2.4 langkah 5 menyimpan openapi.json dari dalam main.ts setiap kali aplikasi dijalankan. Proyek ini memakai skrip terpisah (npm run openapi:export, file scripts/export-openapi.ts) yang membangun aplikasi dalam *preview mode*. Dengan cara ini dokumen dapat diekspor tanpa database dan tanpa menulis file setiap kali server start.

```
$ npm run openapi:export
docs/tugas-2/openapi.{json,yaml}: 7 path
docs/tugas-5/openapi-gateway.{json,yaml}: 15 path
```

8. Verifikasi Swagger

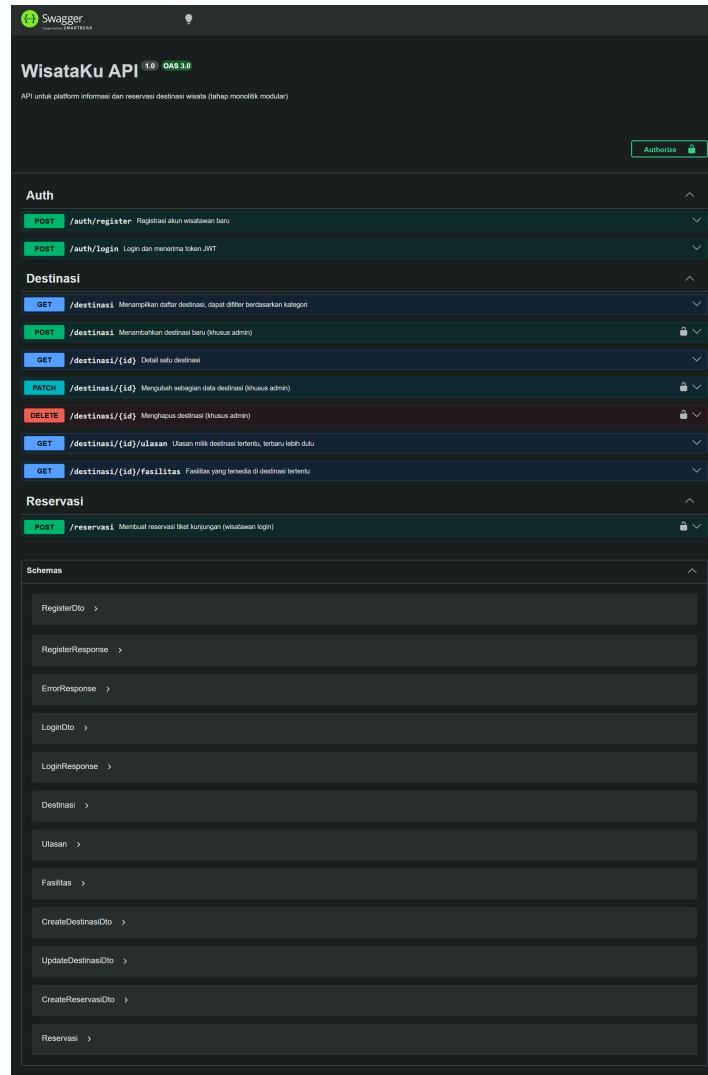
Pemeriksaan dilakukan pada 30 September 2026 terhadap aplikasi hasil build yang dijalankan di mesin pengembangan:

URL	wisataku-api	gateway
GET /api/docs (Swagger UI)	200	200
GET /api/docs-json	200	200
GET /api/docs-yaml	200	200

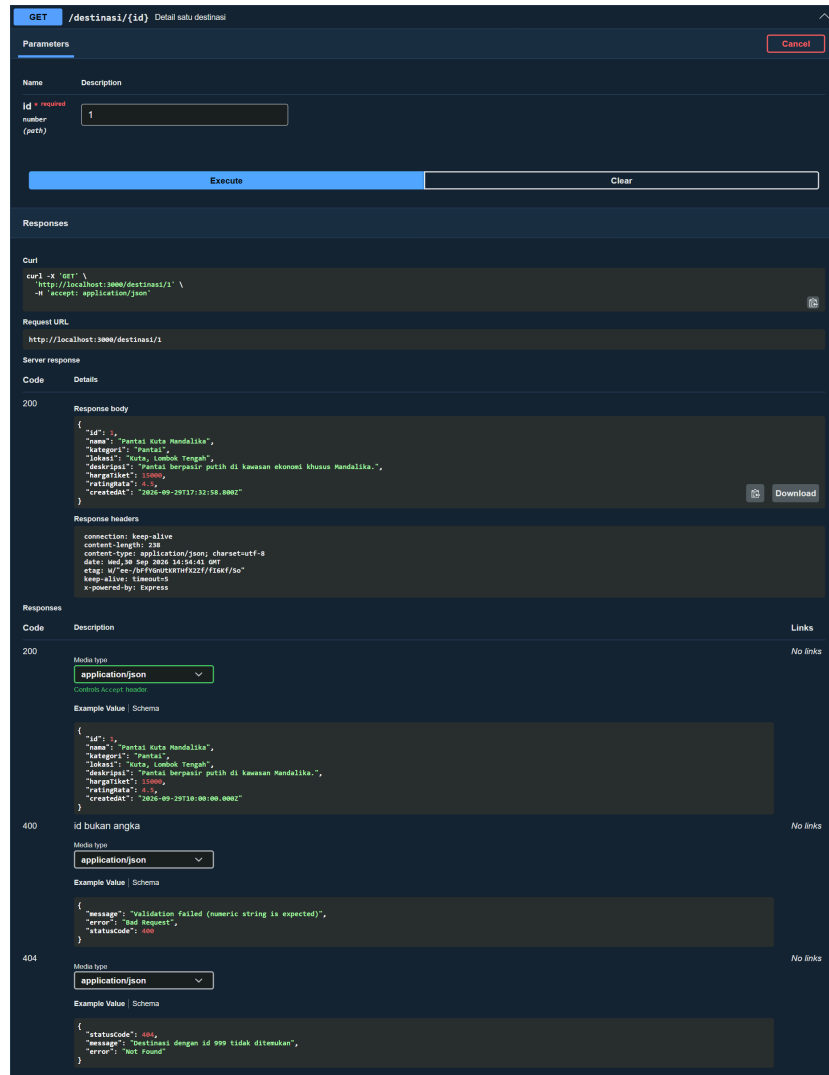
Pemeriksaan kesesuaian	Hasil
Setiap endpoint di OpenAPI benar-benar ada	Smoke test HTTP: monolit 32/32, gateway 37/37
Status code yang dideklarasikan benar-benar dikembalikan	E2E test 41/41 PASS (docs/tugas-4/test-results/e2e-test-2026-09-30.txt)
Body tidak valid ditolak sesuai skema DTO	E2E: register, create destinasi, dan reservasi → 400
Endpoint admin/wisatawan ditandai bearer	4 operasi, sesuai endpoint yang memakai JwtAuthGuard

8.1 Screenshot Swagger UI

Screenshot diambil pada 30 September 2026 dari Swagger UI aplikasi yang berjalan (npm run start:api, http://localhost:3000/api/docs) dengan Chrome yang dikendalikan otomatis (Puppeteer). Seluruh respons pada gambar adalah respons asli server.



Gambar 8.1 Halaman utama Swagger UI. Endpoint yang dilindungi JWT (POST, PATCH, DELETE /destinasi, dan POST /reservasi) ditandai ikon gembok.



Gambar 8.2 "Try it out" GET /destinasi/{id} dengan id = 1. Bagian Server response menampilkan kode 200, body, dan header asli dari server. Di bawahnya terlihat contoh 400 dan 404 yang terdokumentasi.

Pengujian endpoint terproteksi melalui tombol **Authorize** dan "Try it out" POST /destinasi (tanpa token → 401, token wisatawan → 403, token admin → 201) didokumentasikan pada laporan Tugas 4, folder docs/tugas-4/screenshots/.

9. Kesimpulan

1. Endpoint WisataKu mengikuti konvensi REST: kata benda, huruf kecil, hierarki untuk relasi, dan method HTTP sebagai penanda aksi.
2. Setiap endpoint memiliki status code yang semantik dan terdokumentasi (200, 201, 400, 401, 403, 404, 409).

3. DTO menjadi kontrak validasi sekaligus skema dokumentasi, sehingga dokumentasi tidak berbeda dari perilaku API.
4. Dokumen OpenAPI 3.0 tersedia dalam format JSON dan YAML, diekspor langsung dari kode. Swagger UI dapat diakses di `/api/docs`.
5. Swagger UI telah diverifikasi pada aplikasi yang berjalan; screenshot halaman utama dan hasil "Try it out" disertakan pada Bagian 8.1.